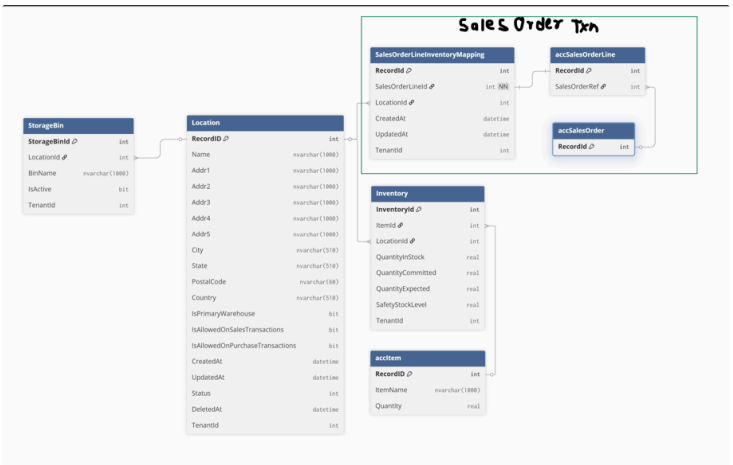
Table Schema RFC

Database structure assuming we will sync Item QoH with QBO

The database structure visualized is as follows :

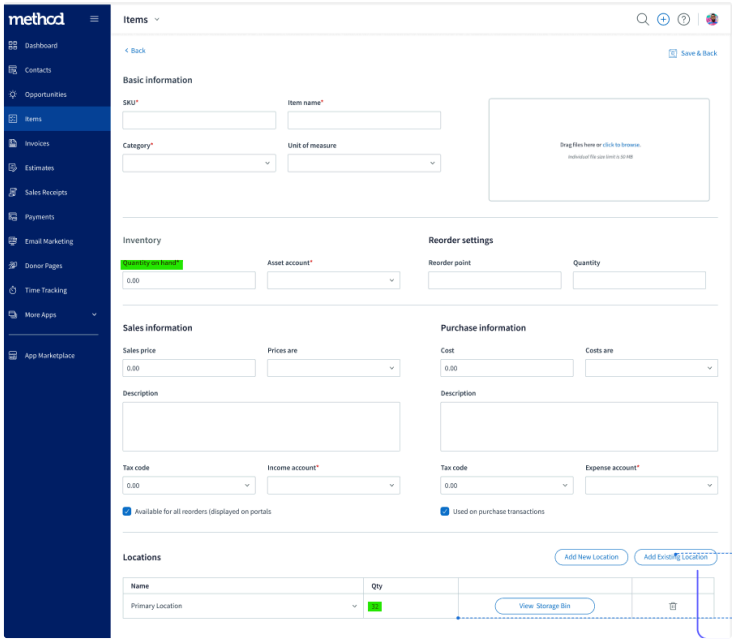


Multi-location Management

Responsibility of each table :

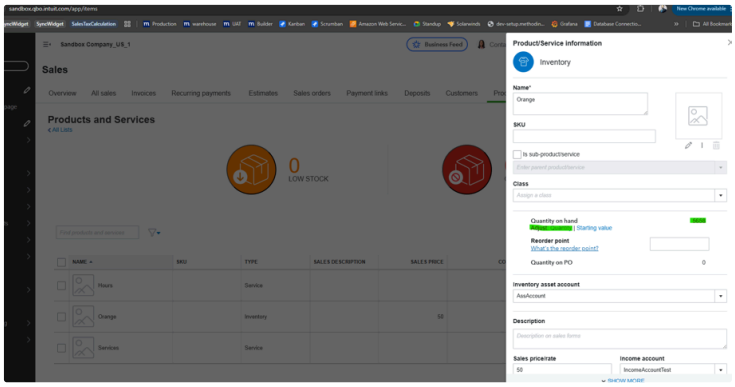
1. Location - Store the warehouse details. This table will always have at least one record. This will be auto set to default WH.
2. accItem - Existing table containing all the items that sync with the accounting platform.
3. Inventory - Mapping table between Location and accItem table. This is the main table that stores the inventory details of all the items that we have. Every time a Sales/Purchase transaction is created that has an inventory item, this table would have and update. Every time a sync with the accounting platform completes, the record with the default WH for the item being processed would be updated.
4. TransactionLineInventoryMapping - 1:1 link between the corresponding transaction line item and Location table. Transaction here could be any A/R or A/P transaction like SalesOrder, PurchaseOrder, SalesReceipt, Invoice, etc.
5. StorageBin - This would not be a part of Multi-location management, will be only creating the bare infrastructure in the database so that we would be able to build on top of the table easily.

Working envisioned :



Referring to the screenshot above, there can be 5 sources that can update quantity on hand for an item :

1. Item screen - Directly updating QuantityOnHand field.
2. Change of quantity at a particular location which then should be reflected in the total quantity on hand.
3. Quantity on hand updated from accounting package - eg. QBO



4. REST API
5. Creation of a transaction utilizing the item (this can be transaction created in QBO but then synced to Method or a transaction created in Method)

All of the 5 scenarios above can be categorized into 2 workflows -

1. Workflow that updates item quantity on hand directly [scenario 1, 3, 4]
2. Workflow that updates item quantity on hand by updating the quantity of that item at one of the locations [scenario 2, 4, 5]

In any workflow, we want the following equation to hold true :
Item QoH = Sum(Item QoH across all locations)

Category 1 workflow (item's QoH updates directly) - Instead of directly updating Item QoH, execute a background process that updates QoH for default location with the delta between new and old values of Item QoH [**default QoH = default QoH + (Item QoH new - Item QoH old)**].
Category 2 workflow (location's QoH updates) - Execute background process that updates Item's QoH by the same delta.

For example -

	Item QoH	Default location QoH for item	Location 2 QoH for item
Before	10	2	8
After	6	-2	8
Direction of calculation	**Item QoH ⟵-- Default QoH**		
Before	10	2	8
After	4	2	2
Direction of calculation	**Item QoH ⟵---------- Location2 QoH**		

Note : Both the above calculation functions should update the respective fields with a lock. That means while updating Item QoH, we will also need to acquire a lock on default location QoH. Similarly, while updating any location's QoH, we would need to acquire a lock on Item QoH to avoid surprising race condition updates. Plan is to use EDA if possible, alternatively follow the lock acquisition workflow above.

For the explanation below, QuantityInStock = QoH.

- Every time a transaction is created, the Inventory.QuantityInStock, Inventory.QuantityCommitted and Inventory.QuantityExpected would get re-calculated and stored in the Inventory table. Since the QuantityInStock for that item at the concerned location is updated, the category 2 workflow should be invoked.
- With every sync from the accounting package, the Quantity from the items table would be changing. If the sync direction is from the accounting package to Method, the items would automatically be added to the default warehouse.

Issues to be resolved - While QBO is disconnected, if the user updates the Item QoH and also has a transaction affecting the Item QoH from QBO, we would not know which workflow from above to follow, category 1 or category 2. Ultimately when the sync runs, it will update the Item QoH first and as the sync progresses, when the transaction syncs, the locations QoH recalculation updates the Item QoH again. In case there is a discrepancy between the Item QoH and the item quantity used on a transaction, this workflow would override the Item QoH to be aligned with transactions rather than retaining the adjust Item QoH. Why this problem comes up is - QBO is an accounting platform and might not care about data consistency between transactions and Item QoH. And since we would not know if the change in Item QoH on QBO happened because of a direct change or indirect change through transactions, this issue becomes challenging to solve and leaves the responsibility of calculating Item QoH on us.

To resolve this we plan to implement stock adjustment transactions and disallow direct change to QoH in Method (through RestAPI, Import of item, Method UI). Doing this allows us to control the direction of change (Item QoH only updates because a transaction affecting that is updated/created/deleted). Workflows to think:

1. Transaction updates in Method
2. Transaction updates in QBO
3. Item QoH unaccounted changes through QBO API.

We would have to set the expectation that inventory management would only work if the account is syncing with QBO. This is because the solution of inventory management is based on Item QoH which in itself is calculated in QBO today. We push transactions created in Method to QBO and let the background sync pull the Item QoH into Method (we do not calculate Item QoH within Method).

If QBO did not allow a direct change to Item QoH through their API, we would not have to look further for a better solution, but they do. So we would have to implement a workflow based on default location adjustment. We have two solutions currently for this :

1. Have a process to tally the Item QoH differences EOD confirming no sync is actively running. Any delta between the total of quantities of each location and Item QoH will be applied to default location.
2. A more complex solution is to over correct first and then get to the correct tally as sync progresses. Any time a transaction syncs from QBO, adjust default location basing the delta on the equation Item QoH = k + i where k is the change resulting from known

transactions and i is the change resulting from unaccounted transactions. Every time we sync any transaction from QBO, we assume Item QoH change was due to k as well as i. After syncing any transaction, if the totals match up, we assume i=0. If they do not, we assume i was not 0 and we apply that delta to default location. Until after the last transaction syncs, if there was no unaccounted Item QoH change, cumulative i should be 0 and the totals should tally. If there was an unaccounted change to Item QoH, it would already be in default WH since we have over corrected through default WH in the first place. Caveat - The quantity for default WH temporarily could be wrong until the sync completes.

Sub-workflows to consider (for simplicity this can be thought from the most granular - a transaction line perspective). In conjunction to above proposed solution 2, we would also need to consider sequence of sync (Item QoH syncs from QBO before transactions sync vs Transactions sync before Item QoH).

1. Transaction line was created in QBO and is syncing with Method for the first time.
2. Transaction line was updated in QBO which is syncing with Method but the transaction already has location management data associated to it.
3. Transaction line was created in Method - this eventually merges in point 2 as we have a “pull-push-pull” on sync.
4. Transaction line was updated in Method - location related quantity would get updated twice (during each pull)
5. Unaccounted change to ItemQoH comes up - We let the sync update Item QoH in Method and do not update anything else. The next time a transaction is synced, this would be auto corrected

Any action below would be taken only after sync. So we would not be updating any data in Method and only take action after synced data comes back from QBO

Scenario 1	**Scenario 2**	**Scenario 3**	**Scenario 4**
Item QoH = Sum (L1...Ln) + L0 8 = 5 + 3 Assume we pulled in a transaction line that changes L2 quantity by +7 We check if after applying this change to L2, does the total add up to current Item QoH If not, we apply the delta to L0 and then update L2. If yes, we only update L2 8 = 12 + (-4) At this point, assume we pulled in a transaction line that changes L4 by +3 We check if after applying this change to L4, does the total add up to current Item QoH If not, we apply the delta to L0 and then update L4. If yes, we only update L4 8 = 15 + (-7) At this point, assume we pulled in Item changes that change the Item QoH by +10	Item QoH = Sum (L1...Ln) + L0 105 = 90 + 15 Assume we pulled in a transaction line that changes L2 quantity by -18 We check if after applying this change to L2, does the total add up to current Item QoH If not, we apply the delta to L0 and then update L2. If yes, we only update L2 105 = 72 + (+33) At this point, assume we pulled in a transaction line that changes L4 by -3 We check if after applying this change to L4, does the total add up to current Item QoH If not, we apply the delta to L0 and then update L4. If yes, we only update L4 105 = 69 + (+36) At this point, assume we pulled in transaction line that changes L0 by -14	Item QoH = Sum (L1...Ln) + L0 115 = 0 + 105 Assume we pulled in a transaction line that changes L0 quantity by -18 We check if currently the totals add up Item QoH = Sum (L1...Ln) + L0. If yes, we do not update anything. If not, we update L0 only by the difference between Item QoH - Sum (L1...Ln) 115 = 0 + (+115) At this point, assume we pulled in a transaction line that changes L4 by -3 We check if after applying this change to L4, does the total add up to current Item QoH If not, we apply the delta to L0 and then update L4. If yes, we only update L4 115 = -3 + (+112) At this point, assume we pulled in transaction line that changes L0 by -14	Item QoH = Sum (L1...Ln) + L0 115 = 0 + 105 Assume we pulled in a transaction line that changes L2 quantity by -4 We check if after applying this change to L2, does the total add up to current Item QoH If not, we apply the delta to L0 and then update L2. If yes, we only update L2 115 = -4 + (119) At this point, assume we pulled in Item changes that change the Item QoH by +7 We check if after applying this change to Item QoH, does the total add up to new Item QoH = Sum (L1...Ln) + L0 If not, we apply the delta to L0 and Item QoH. If yes, we only update Item QoH 122 = -4 + (+126)

We check if after applying this change to Item QoH, does the total add up to new Item QoH = Sum (L1...Ln) + L0 If not, we apply the delta to L0 and Item QoH. If yes, we only update Item QoH 18 = 12 + (+6) At this point, assume we pulled in transaction line that changes L0 by -15 We check if currently the totals add up Item QoH = Sum (L1...Ln) + L0. If yes, we do not update anything. If not, we update L0 only by the difference between Item QoH - Sum (L1...Ln) 18 = 12 + (+6)	We check if currently the totals add up Item QoH = Sum (L1...Ln) + L0. If yes, we do not update anything. If not, we update L0 only by the difference between Item QoH - Sum (L1...Ln) 105 = 69 + (+36) At this point, assume we pulled in Item changes that change the Item QoH by -25 We check if after applying this change to Item QoH, does the total add up to new Item QoH = Sum (L1...Ln) + L0 If not, we apply the delta to L0 and Item QoH. If yes, we only update Item QoH 80 = 69 + (+11)	We check if currently the totals add up Item QoH = Sum (L1...Ln) + L0. If yes, we do not update anything. If not, we update L0 only by the difference between Item QoH - Sum (L1...Ln) 115 = -3 + (+112) At this point, assume we pulled in Item changes that change the Item QoH by -3 We check if after applying this change to Item QoH, does the total add up to new Item QoH = Sum (L1...Ln) + L0 If not, we apply the delta to L0 and Item QoH. If yes, we only update Item QoH 112 = -3 + (+115)	

Discussions related to exploring Inventory management as a separate entity, not dependent on syncing Item QoH with QBO

- The items will continue syncing with QuickBooks Online (QBO) even if you maintain separate “Quantity on Hand (QoH)” and “Item Total” fields for inventory management.
- All transactions that sync with QBO will still be leveraged for inventory management, regardless of the separate fields of QoH and Item total of inventory management.

Considering the above, we want to decide on the following -

1. Should Method or QBO be the source of truth (SOT) for IMS?
2. If Method is the SOT, do we decouple QOH from QBO?
3. What reconciliation tools/logs/audits must we build to maintain trust?
4. When Katana is connected, does it become the new SOT?
5. What happens when conflicting data flows in from Katana and QBO?

Decoupled Item Quantity on Hand that syncs with QBO and Item total for IMS

From engineering perspective, going back from this decision would be very difficult since it would require all of our customers consolidating the two quantities on hand for all items or us providing a generic consolidation which could break the data in a few accounts since all accounts might have a different workflow.

Advantages	Disadvantages
Allows IMS solution to be used independent of sync status - useful in cases where customer has disconnected from QBO, allows IMS to be used for all accounting platforms	User will have to manually consolidate the Quantity that syncs with QBO vs the Item total field that is being used for the IMS
Avoids complexities involving sync sequences where Item updates sync after dependent transactions	Not a seamless syncing solution. Also closes the path for having a 3 way sync of Item quantity between an external IMS like Katana, Method and QBO

	Requires some re-designing from product/design team to decide if we want to show two QoH values on the item screen and if so, rules around when to show inventory management value vs QBO value. QBO value is mandatory on Item screen - we might need a separate section to add inventory management value

Method is SOT for IMS

From engineering perspective, we can implement the calculation logic such that it is decoupled and can be “replaced” with an external IMS solution. So technically, we can walk back on this decision although, the data that has been entered already keeping Method as SOT would need to comply with the external IMS solution (ex. Method items that sync with QBO as inventory but Katana needs all items that sync with QBO as non-inventory)

Advantages	Disadvantages
We can define rules/workflows as we want, we can update them as required.	The calculation complexities would be the responsibility of Method taking our focus away from us being a CRM company to additionally being an Inventory Management company.
If we want to go with any other inventory management software tomorrow instead of Katana, that allows Method to be the SOT and only use the external software for substituting inventory calculations	The rule logic also needs to be implemented by Method. (Ex. do we allow updates from QBO to reflect the Inventory in case the transactions is invoiced or do we only sync the transaction but not let it reflect the inventory once already invoiced)

Reconciliation tools/logs/audits to maintain trust

Option 1: Enhanced Audit Trail Only

Advantages	Disadvantages
Uses existing Elasticsearch infrastructure - no new systems needed, leverages current investment	Generic key-value pairs require complex parsing - every inventory query needs JSON manipulation, slower performance at scale
Minimal development effort - 1-2 sprints to add inventory filters and UI views	Cannot track transfer orders - no way to link movements between locations, making multi-location reconciliation impossible
Single audit system for all operations - maintains consistency across Method platform	Limited to reactive investigation - can only search after problems occur, no proactive variance detection

Option 2: Hybrid - Audit Trail + Inventory Ledger

Advantages	Disadvantages
Purpose-built for inventory reconciliation - tracks quantity states (before/after), transfer orders, and multi-source conflicts	Additional infrastructure to maintain - separate table and processes, though uses same database and event system
Enables proactive trust building - automated variance detection, source attribution for Method/QBO/Katana conflicts, complete forensic tracking	Implementation time - more initial development than audit trail enhancement alone
Scales with business complexity - supports multi-location transfers, handles 3-way sync scenarios, reduces investigation time from hours to seconds	Duplicate event storage - inventory events recorded in both audit trail and ledger, though storage cost is minimal

Why Hybrid: Audit trail alone cannot provide the structured data needed for multi-location reconciliation and transfer tracking. The ledger enables the trust features customers need while keeping audit trail unchanged for compliance.

Detailed Analysis: Inventory Tracking Comparison Document

Once Katana is connected, it becomes the new SOT

Advantages	Disadvantages
From Method's perspective, we only would have to push data to Katana and pull the dependent data from Katana avoiding the calculation complexities involved in Inventory Management, allowing the Method to focus on CRM areas.	We would have to update the workflow from Method being the SOT to Katana being the SOT, which adds complexity to define what should happen with the data that has synced with QBO as inventory items, actively being used in Method's IMS. This complexity because Katana syncs items as non-inventory to QBO

Problematic scenarios even when separate QoH quantities are implemented

#	Problematic Workflow	Mechanism	Issues
1	**QBO Invoice Update Overwrites Inventory in Method (When Katana is SOT)**	QBO invoice update syncs to Method, but Katana overwrites Method's inventory during the next sync.	Loss of QBO-driven updates, data integrity issues, user confusion. Proposed IMS Architecture Matrix (Product Perspective)
2	**Double Counting Inventory Adjustments**	Sales order in Method pushes to Katana (inventory reserved), then invoice in QBO syncs to Method (inventory adjusted again).	Inventory reduced twice, negative/inaccurate stock levels, manual reconciliation needed. Comprehensive Workflows for Solutioning in Multi-Location Management Systems
3	**Unaccounted Direct QoH Edits**	Manual QoH edits in Method are overwritten by Katana during sync, with no audit trail or notification.	Manual adjustments lost, trust issues, lack of transparency.
4	**Conflicting Data from QBO and Katana**	QBO and Katana both sync inventory adjustments to Method, causing conflicts.	Data may flip-flop or become inconsistent, unclear which system is accurate. Proposed IMS Architecture Matrix (Product Perspective). For this, we can keep "Inventory adjustment" transaction as QBO and "Stock adjustment" transaction as Katana oriented.
5	**Deletion of Transactions Not Propagated**	Deletion in QBO or Method is not synced to Katana, so inventory reductions remain.	Inventory discrepancies persist, manual intervention required. Comprehensive Workflows for Solutioning in Multi-Location Management Systems
6	**Change in synced Item type with QBO**	Update from inventory to non-inventory type using QBO for item that has synced to Katana	Inventory discrepancies persist, manual intervention required.

Finalized database structure keeping QBO QoH and Method Inventory QoH separate

A Free Database Designer for Developers and Analysts

```
Table invLocation {
  RecordId                      int        [pk]
  Name                          nvarchar   [note: "length 1000"]
  Addr1                         nvarchar   [note: "length 1000"]
  Addr2                         nvarchar   [note: "length 1000"]
  Addr3                         nvarchar   [note: "length 1000"]
  Addr4                         nvarchar   [note: "length 1000"]
  Addr5                         nvarchar   [note: "length 1000"]
  City                          nvarchar   [note: "length 510"]
  State                         nvarchar   [note: "length 510"]
  PostalCode                    nvarchar   [note: "length 60"]
  Country                       nvarchar   [note: "length 510"]
  IsPrimaryWarehouse            bit
  IsAllowedOnSalesTransactions  bit
  IsAllowedOnPurchaseTransactions bit
  CreatedAt                     datetime
  UpdatedAt                     datetime
  Status                        int
  DeletedAt                     datetime
  TenantId                      int
}

Table invStorageBin {
  StorageBinId       int        [pk]
  LocationId         int        [ref: > invLocation.RecordId]
  BinName            nvarchar   [note: "length 1000"]
  IsActive           bit
  TenantId           int
}

Table acctItem {
  RecordID           int        [pk]
  ItemName           nvarchar   [note: "length 1000"]
  QuantityInStock    real
  QuantityOnHand     real

  Note: '''
  Trigger: On insert, to insert record into Inventory table to link default location (get current default location from location table) with QuantityInStock = QuantityOnHand
  '''
}

Table invInventory {
  InventoryId        int        [pk]
  ItemId             int        [ref: > acctItem.RecordID]
  LocationId         int        [ref: > invLocation.RecordId]
  QuantityInStock    real
  QuantityCommitted  real
  QuantityExpected   real
  SafetyStockLevel   real
  TenantId           int

  Note: '''
  Trigger: On insert/update, to calculate total across all locations of the item QuantityInStock and update QuantityInStock in accItem
  (will re-trigger accItem update only once since accItem trigger is only on insert. This retrigger will update accItem.QuantityInStock)
  '''
}

Table accSalesOrder {
  RecordId           int        [pk]
  // other fields
}

Table accSalesOrderLine {
  RecordId           int        [pk]
  SalesOrderRef      int        [ref: > accSalesOrder.RecordId]
  Quantity           real
  // other fields

  Note: '''
  Trigger: On update of accSalesOrderLine.Quantity, to update Inventory table Item QuantityCommitted to account for accSalesOrderLine.Quantity change
  '''
}

Table invSalesOrderLineInventoryMapping {
  RecordId           int        [pk]
  SalesOrderLineId   int        [ref: > accSalesOrderLine.RecordId]
  LocationId         int        [ref: > invLocation.RecordId]
```

```
  CreatedAt          datetime
  UpdatedAt          datetime
  TenantId           int

  Note: '''
  Trigger: On insert, to update Inventory table Item QuantityCommitted to account for accSalesOrderLine.Quantity change
  Trigger: On update (location change), to update Inventory table by reducing Item QuantityCommitted from one location and increasing Item QuantityCommitted in the selected location by accSalesOrderLine.Quantity change
  '''
}

Table accPurchaseOrder {
  RecordId           int        [pk]
  // other fields
}

Table accPurchaseOrderLine {
  RecordId           int        [pk]
  PurchaseOrderRef   int        [ref: > accPurchaseOrder.RecordId]
  Quantity           real
  // other fields
  Note: '''
  Trigger: On update of accPurchaseOrderLine.Quantity, to update Inventory table Item QuantityExpected to account for accPurchaseOrderLine.Quantity change
  '''
}

Table invPurchaseOrderLineInventoryMapping {
  RecordId              int     [pk]
  PurchaseOrderLineId   int     [ref: > accPurchaseOrderLine.RecordId]
  LocationId            int     [ref: > invLocation.RecordId]
  CreatedAt             datetime
  UpdatedAt             datetime
  TenantId              int

  Note: '''
  Trigger: On insert, to update Inventory table Item QuantityExpected to account for accPurchaseOrderLine.Quantity change
  Trigger: On update (location change), to update Inventory table by reducing Item QuantityExpected from one location and increasing Item QuantityExpected in the selected location by accPurchaseOrderLine.Quantity change
  '''
}

Table accInvoice {
  RecordId           int        [pk]
  // other fields
}

Table accInvoiceLine {
  RecordId           int        [pk]
  InvoiceRef         int        [ref: > accInvoice.RecordId]
  Quantity           real
  // other fields
  Note: '''
  Trigger: On update of accInvoiceLine.Quantity, to update Inventory table Item QuantityInStock to account for accInvoiceLine.Quantity change
  '''
}

Table invInvoiceLineInventoryMapping {
  RecordId           int        [pk]
  InvoiceLineId   int           [ref: > accInvoiceLine.RecordId]
  LocationId         int        [ref: > invLocation.RecordId]
  CreatedAt          datetime
  UpdatedAt          datetime
  TenantId           int

  Note: '''
  Trigger: On insert, to update Inventory table Item QuantityInStock to account for accInvoiceLine.Quantity change
  Trigger: On update (location change), to update Inventory table by increasing Item QuantityInStock from one location and reducing Item QuantityInStock in the selected location by accInvoiceLine.Quantity change
  '''
}

Table accSalesReceipt {
  RecordId           int        [pk]
  // other fields
}

Table accSalesReceiptLine {
  RecordId           int        [pk]
  SalesReceiptRef    int        [ref: > accSalesReceipt.RecordId]
  Quantity           real
  // other fields
```

```

  Note: '''
  Trigger: On update of accSalesReceiptLine.Quantity, to update Inventory table Item QuantityInStock to account for accSalesReceiptLine.Quantity change
  '''
}

Table invSalesReceiptLineInventoryMapping {
  RecordId           int        [pk]
  SalesReceiptLineId   int      [ref: > accSalesReceiptLine.RecordId]
  LocationId         int        [ref: > invLocation.RecordId]
  CreatedAt          datetime
  UpdatedAt          datetime
  TenantId           int

  Note: '''
  Trigger: On insert, to update Inventory table Item QuantityInStock to account for accSalesReceiptLine.Quantity change
  Trigger: On update (location change), to update Inventory table by increasing Item QuantityInStock from one location and reducing Item QuantityInStock in the selected location by accSalesReceiptLine.Quantity change
  '''
}
Table invStockAdjustment {
  RecordId                  int        [pk]
  LocationId                int        [ref: > invLocation.RecordId]
  StockAdjustmentDate       datetime
  StockAdjustmentRefNumber  nvarchar   [note: "length 500"]
  StockAdjustmentReason     nvarchar   [note: "MAX"]
  CreatedAt                 datetime
  UpdatedAt                 datetime
  TenantId                  int

  Note: '''
  Trigger: On update (Location change), to update Inventory table by changing Item QuantityInStock for each item in invStockAdjustmentLine from previous invStockAdjustment.LocationId to new invStockAdjustment.LocationId
  '''
}

Table invStockAdjustmentLine {
  RecordId                  int        [pk]
  StockAdjustmentRef        int        [ref: > invStockAdjustment.RecordId]
  ItemId                    int        [ref: > acctItem.RecordID]
  Quantity                  real
  CreatedAt                 datetime
  UpdatedAt                 datetime
  TenantId                  int

  Note: '''
  Trigger: On insert, to update Inventory table Item QuantityInStock to account for invStockAdjustmentLine.Quantity change at invStockAdjustment.LocationId
  Trigger: On update (Quantity change), to update Inventory table by changing Item QuantityInStock from invStockAdjustment.LocationId
  '''
}
```

Discussion related to SO/PO fulfillment flow

We want to decide on the following:

1. Which fulfillment version should we build for MVP — V1 (partial-per-line delivery) or V2 (whole-line only)?
2. Should inventory and accounting flows be kept fully separate, or should we tie delivery/receiving to invoicing/billing events?
3. What are the sync implications (if any) of either decision above with QBO?
4. How much do we want to mimic Katana? At what point do we start becoming a competitor vs a partner?
5. If we allow undoing delivery/receiving of items in SO/PO, how should this be implemented and are there any sync implications with QBO we should be aware of?
6. Should SO/PO be editable after creation?
7. Do we allow different taxation for the parts of the same SO?

Refer to this link for answers to above questions.